

Документация проекта GraphRAGon с поддержкой Graph of Thoughts

Utopialvo

26 июня 2026 г.

Содержание

1	Введение	3
2	Архитектура системы	3
3	Модель графа знаний	4
4	Обработка входного текста	5
5	Разрешение конфликтов	6
5.1	Обнаружение конфликтов	6
5.2	Итеративное разрешение	6
6	Вопросно-ответная система	7
6.1	Простой режим (<code>use_got=False</code>)	7
6.2	Graph of Thoughts (<code>use_got=True</code>)	8
7	Хранилище Memgraph	8
7.1	Транзакции и блокировки	8
7.2	Работа с пассажами	9
7.3	Отношения и <code>based_on</code>	9
7.4	Векторный поиск	9
7.5	Снепшоты и очистка	9
8	Клиенты LLM и эмбедингов	10
8.1	LLMClient	10
8.2	EmbeddingClient	10
9	Утилиты безопасного парсинга JSON	10
10	Особенности реализации и принятые решения	11
11	Заключение	12

1 Введение

Graph RAG (Retrieval-Augmented Generation) — это подход к вопросно-ответным системам, который объединяет поиск релевантной информации в графе знаний и генерацию ответа с помощью большой языковой модели (LLM). В отличие от классических RAG-систем, опирающихся на векторный поиск по коллекции неструктурированных документов, графовая версия хранит извлечённые факты в виде сущностей и связей между ними, что позволяет моделировать семантические отношения и давать более точные, контекстные ответы. Под графом знаний здесь понимается структура, состоящая из узлов-сущностей и рёбер-отношений, каждое из которых снабжено указанием на исходный текстовый фрагмент (пассаж), обеспечивая прослеживаемость (provenance). Векторные представления (эмбединги) сущностей, полученные с помощью модели sentence-transformers, используются для быстрого поиска семантически близких узлов через встроенный в графовую СУБД Memgraph векторный индекс.

Данный проект реализует полный пайплайн:

- Загрузка произвольного текста, его нарезка на пересекающиеся чанки.
- Извлечение именованных сущностей и связей из каждого чанка с помощью локальной LLM.
- Сохранение сущностей, связей и исходных фрагментов в графовой СУБД Memgraph.
- Разрешение конфликтов между противоречащими друг другу отношениями (например, когда из разных фрагментов один и тот же субъект совершает взаимоисключающие действия).
- Ответ на пользовательский вопрос с использованием техники *Graph of Thoughts* (GoT): вопрос дробится на подвопросы, каждый обрабатывается отдельно, и ответы агрегируются в единый связный текст.

Система ориентирована на исследовательские и прототипные задачи, демонстрируя возможность построения полноценного графового RAG на локальном оборудовании с открытыми моделями. Ключевая особенность — акцент на практические аспекты: кэширование дорогостоящих операций, устойчивость к ошибкам модели, потокобезопасность при параллельной обработке и автоматическое создание резервных копий графа.

2 Архитектура системы

Центральным элементом является класс **GraphRAG**, который координирует работу всех подсистем. Основные компоненты и их взаимодействие таковы:

- **LLMClient** — обеспечивает обмен сообщениями с языковой моделью через OpenAI-совместимый HTTP-интерфейс. Поддерживает кэширование ответов в SQLite, автоматическую очистку старых записей и повторные попытки при сетевых ошибках.
- **EmbeddingClient** — вычисляет векторные представления текстов с помощью библиотеки sentence-transformers. Имеет два режима: одиночный и пакетный, с собственным кэш-слоем на SQLite.

- **MemgraphStore** — прослойка для работы с графовой базой данных Memgraph. Содержит методы для создания и удаления узлов и связей, управления транзакциями, векторного поиска и создания снапшотов.
- **LLMEntityExtractor** и **RelationExtractor** — специализированные экстракторы, использующие LLM для извлечения сущностей и отношений из заданного текста. Реализуют fallback-стратегию: при неудачном разборе основного промпта применяется упрощённый.
- **ConflictResolver** — анализирует пары отношений с одинаковыми субъектом и объектом на предмет противоречий и разрешает их (удаление, оставление одного или слияние). Реализован итеративный алгоритм, гарантирующий консистентность после каждого шага.
- **HistoryManager** — хранит историю диалога в оперативной памяти (циклическая очередь фиксированного размера) и предоставляет текстовое представление для включения в промпты.

Все компоненты слабо связаны: каждый модуль решает свою задачу и взаимодействует с другими через чётко определённые интерфейсы (методы и датаклассы Pydantic). Конфигурация системы вынесена в YAML-файл и загружается в датакласс **GraphRAGConfig**, что позволяет легко адаптировать систему под различное окружение.

Потоки данных при обработке текста: текст → чанки → для каждого чанка последовательно вызываются экстракторы (сущности → отношения) → результаты сохраняются в Memgraph. Затем для групп отношений с одинаковыми головой и хвостом запускается конфликт-резольвер. При ответе на вопрос выполняется обратный процесс: векторный поиск ближайших сущностей, извлечение их контекста (пассажи, соседние сущности) и формирование промпта для генерации ответа.

3 Модель графа знаний

Граф в Memgraph организован как трёхуровневая память, сочетающая схематические, фактологические и источниковедческие аспекты. Введём основные понятия.

Сущность (Entity) — это уникальный объект реального мира или абстракция, извлечённая из текста. Каждая сущность имеет имя (текст), тип (например, PERSON, LOCATION, OBJECT, ACTION, TIME) и векторное представление (embedding), вычисленное по её имени. Тип сущности связывается с узлом схемы (Schema) отношением **INSTANCE_OF**, что позволяет в дальнейшем проводить семантическую фильтрацию и обобщение. Векторное представление — это плотный числовой вектор размерности, определяемой моделью эмбедингов (по умолчанию 384 для **all-MiniLM-L6-v2**), который отражает семантику имени сущности. Он используется для поиска близких по смыслу сущностей через векторный индекс.

Пассаж (Passage) — это исходный фрагмент текста (чанк), из которого были выделены сущности и отношения. Пассаж хранит полный текст и уникальный идентификатор (UUID). Связь **MENTIONED_IN** соединяет сущность с пассажем, указывая, что данная сущность упоминалась в этом фрагменте. Это обеспечивает прослеживаемость: для любого факта можно найти его первоисточник. При повторном обнаружении той же сущности в том же пассаже связь не дублируется (используется **MERGE**).

Отношение (Relation) — направленная связь между двумя сущностями с меткой `RELATION` и атрибутами: `type` (глагольная фраза, например, “рубил”, “растопила”) и `based_on` (список идентификаторов пассажей, на основе которых было извлечено это отношение). Такой дизайн позволяет хранить множество доказательств для одного и того же факта: если отношение повторяется в разных чанках, их идентификаторы накапливаются в `based_on`, и при необходимости можно проследить все источники. При добавлении нового пассажа к существующему отношению используется условная конструкция в Cypher-запросе: `ON CREATE SET r.based_on = [$passage_id]; ON MATCH SET r.based_on = CASE WHEN $passage_id NOT IN r.based_on THEN r.based_on + [$passage_id] ELSE r.based_on END`. Это гарантирует отсутствие дубликатов и идемпотентность операций.

Схема (Schema) — узлы, описывающие типы сущностей (`entity_type`) и типы отношений (`relation_type`). Создаются автоматически при первом упоминании типа, что делает граф самоописывающим и облегчает интеграцию с внешними онтологиями.

Векторный индекс строится по полю `embedding` узлов `Entity` с использованием встроенного в Memgraph механизма на базе библиотеки USearch. При инициализации хранилища проверяется существование индекса с именем `entity_embeddings`; если размерность не совпадает с текущей моделью эмбедингов, индекс пересоздаётся. Это позволяет безболезненно менять модель эмбедингов, не разрушая граф.

4 Обработка входного текста

Метод `process_text` принимает произвольный текст и опциональные параметры чанкинга. Разберём этапы подробно.

Чанкинг — процесс разбиения длинного текста на перекрывающиеся фрагменты (чанки) заданного размера (в словах). Перекрытие необходимо, чтобы не терять контекст на границах: отношения могут связывать сущности, расположенные в разных чанках, и без перекрытия часть связей могла бы быть упущена. Размер чанка и перекрытие задаются в конфигурации (по умолчанию 500 слов с перекрытием 100 слов). Шаг сдвига вычисляется как `chunk_size - overlap`. Каждый чанк сохраняется в графе как узел `Passage` с уникальным UUID, возвращаемым методом `add_passage`.

Далее для каждого чанка в отдельной транзакции выполняются следующие шаги:

- Вызывается `LLMEntityExtractor.extract`, который отправляет LLM промпт с просьбой извлечь сущности в формате JSON-массива объектов `{ "text": "...", "type": "..." }`. Для повышения надёжности сначала используется подробный промпт, а при неудачном разборе ответа (не удалось извлечь JSON) — упрощённый fallback-промпт. Ответ модели обрабатывается функцией `safe_parse_json`, которая умеет извлекать JSON даже из текста, обёрнутого в markdown-блоки или смешанного с пояснениями.
- Полученные сущности добавляются в граф через `store.add_entity`. Если сущность с таким именем уже существует, проверяется её тип: если он был `UNKNOWN`, а новый тип отличается, происходит обновление. Связь с пассажем `MENTIONED_IN` создаётся операцией `MERGE`, чтобы избежать дублирования рёбер при повторных вызовах. Для новой сущности также вычисляется эмбединг (если доступен `embedding_client`) и создаётся связь со схемой.

- Затем `RelationExtractor.extract` принимает список извлечённых сущностей и текст чанка, формирует промпт с перечислением сущностей и получает от модели JSON-массив троек `head`, `relation`, `tail`. Здесь также используется `fallback`-промпт. Каждое отношение валидируется через `Pydantic` и добавляется в граф с идентификатором текущего пассажира.
- При успешном завершении всех операций транзакция коммитится; если на любом этапе возникло исключение (например, ошибка сети, некорректный ответ модели) — транзакция откатывается, и чанк пропускается. Это обеспечивает атомарность: граф не загрязняется частично обработанными данными.

После обработки всех чанков, если включено разрешение конфликтов (`resolve_conflicts=True`), вызывается `ConflictResolver.detect_and_resolve`. Затем создаётся снимок базы данных `Memgraph`, фиксирующий текущее состояние графа.

Все метрики (количество сущностей, отношений, пассажиров и время обработки) накапливаются в атрибуте `metrics` экземпляра `GraphRAG`, что позволяет отслеживать производительность и объём извлечённых знаний.

5 Разрешение конфликтов

Проблема конфликтов возникает из-за того, что разные чанки могут описывать одни и те же сущности, но приписывать им противоречивые действия. Например, в одном фрагменте «Иван купил молоко», а в другом «Иван продал молоко». Оставлять оба отношения в графе нельзя — они противоречат друг другу и сделают дальнейшие ответы несогласованными. Задача конфликт-резольвера — обнаружить такие пары и принять решение: оставить одно, удалить оба и заменить на новое (слияние) или, если конфликта нет, сохранить оба.

5.1 Обнаружение конфликтов

Класс `ConflictResolver` агрегирует все отношения, сгруппированные по паре (`head`, `tail`). Для каждой группы, где больше одного отношения, выполняется попарное сравнение всех комбинаций с разными названиями. Для каждой пары извлекается текст первого связанного пассажира (по первому идентификатору из списка `based_on`), после чего формируется специализированный промпт:

Ты — система разрешения конфликтов в графе знаний. Даны две тройки (субъект, отношение, объект) ... Определи, являются ли эти отношения конфликтующими ... Верни ТОЛЬКО JSON с полями: `conflict`, `resolution`, `new_relation`, `explanation`.

Модель должна вернуть JSON, где `resolution` принимает одно из трёх значений: `"keep_first"`, `"keep_second"` или `"merge"` (при `conflict=true`). Если ответ не удастся распарсить или значение `resolution` некорректно, выполняется до `max_retries` повторных попыток.

5.2 Итеративное разрешение

Ключевой методологический нюанс — применение решений. Ранняя версия модуля сначала собирала все конфликты в общий список, а затем последовательно применяла их, что приводило к ошибкам: после удаления одного отношения следующие

шаги могли ссылаться на уже несуществующие рёбра. Текущая реализация использует итеративный подход внутри каждой группы (`_resolve_group`).

Алгоритм для одной пары (`head`, `tail`):

1. Строится рабочий список отношений — копий словарей с `relation` и `based_on`.
2. Пока в списке больше одного элемента:
 - (a) Для каждой несовпадающей пары вызывается `_get_resolution`.
 - (b) Если конфликт обнаружен:
 - `"keep_first"` — удаляется второе отношение из графа и из рабочего списка.
 - `"keep_second"` — удаляется первое.
 - `"merge"` — оба исходных отношения удаляются, создаётся новое с именем `new_relation` (или “связан” по умолчанию). В его `based_on` попадают все идентификаторы пассажиров обоих старых отношений. В рабочем списке вместо двух удалённых появляется одно новое отношение.
 - (c) После любого изменения списка (добавления/удаления) цикл начинается заново: повторно перебираются все пары с новым составом.
3. Если за полный проход не найдено ни одного конфликта, итерации прекращаются.

Такой подход гарантирует, что каждое последующее решение принимается с учётом уже выполненных изменений, и граф остаётся консистентным. При слиянии особенно важно сохранить все ссылки на пассажи, чтобы новое отношение имело полную доказательную базу.

6 Вопросно-ответная система

Основной метод `ask` реализует два режима: простой (без декомпозиции) и с Graph of Thoughts. Рассмотрим их подробно.

6.1 Простой режим (`use_got=False`)

При отключённом GoT выполняются следующие шаги:

- Векторный поиск: пользовательский вопрос преобразуется в эмбединг, и в графе ищутся ближайшие сущности (по косинусному расстоянию через векторный индекс). Количество результатов ограничено параметром `top_k`.
- Для каждой найденной сущности извлекаются связанные пассажи (до двух) и, если включён случайный обход (`random walk`), дополнительно получается контекст соседних сущностей. Механизм обхода: для каждого отношения сущности (в обе стороны) извлекается соседняя сущность, и для неё берётся один пассаж. Это расширяет контекст за пределы прямых упоминаний.
- Все собранные фрагменты текста объединяются, дубликаты удаляются с сохранением порядка, и результат урезается до `max_context_length` символов.
- Формируется промпт, включающий историю диалога (текстовое представление из `HistoryManager`), полученный контекст и вопрос. Ответ генерируется LLM.

6.2 Graph of Thoughts (use_got=True)

Техника Graph of Thoughts (GoT) повышает качество ответов на сложные вопросы путём их декомпозиции на более простые подвопросы, независимого поиска ответов на них и последующей агрегации. Реализация включает три этапа:

1. **Декомпозиция.** Модели отправляется промпт с просьбой разбить исходный вопрос на список подвопросов в виде JSON-массива строк. Пример ожидаемого ответа: ["Кто участвовал в действиях? "Какие действия выполнял субъект А?"]. Если модель возвращает некорректный JSON, в качестве единственного подвопроса берётся исходный вопрос — это fallback, гарантирующий, что система не остановится.
2. **Ответы на подвопросы.** Если включён параллельный режим (`got_parallel`) и подвопросов больше одного, для каждого создаётся отдельный поток в пуле `ThreadPoolExecutor`. Поскольку основной `LLMClient` не рассчитан на одновременное использование из нескольких потоков (может вызвать гонки в HTTP-сессиях), на каждый поток создаётся собственный экземпляр `LLMClient` с идентичной конфигурацией. В каждом потоке вызывается метод `_answer_subquestion`, который извлекает контекст аналогично простому режиму (векторный поиск + обход) и отправляет модели промпт с подвопросом. При возникновении ошибки подвопрос получает пустой ответ, что позволяет агрегации продолжить работу.
3. **Агрегация.** Все непустые ответы объединяются в список, и вместе с историей диалога и исходным вопросом формируется финальный промпт. Модель генерирует связный итоговый ответ, синтезируя информацию из ответов на подвопросы.

История диалога ведётся в `HistoryManager` — циклической очереди фиксированной длины (по умолчанию 10 записей). Каждая запись — кортеж (вопрос, ответ). Метод `get_history_as_text` преобразует историю в форматированный текст, который добавляется в промпты как при простом режиме, так и при агрегации. Это позволяет системе поддерживать контекст многократного диалога и давать более связные ответы.

7 Хранилище Memgraph

7.1 Транзакции и блокировки

Все публичные методы `MemgraphStore` защищены одним общим замком (`threading.Lock`). Это решение продиктовано тем, что стандартный клиент `GQLAlchemy` не гарантирует потокобезопасность: параллельные запросы из разных потоков могут перемешиваться, нарушая целостность операций. Блокировка захватывается перед выполнением любого запроса к базе и освобождается после его завершения, что сериализует доступ. Хотя это потенциально снижает параллелизм при интенсивной работе с графом, для прототипа такой подход оправдан, поскольку основные задержки приходятся на LLM, а не на операции с данными.

Транзакции управляются явно методами `begin`, `commit` и `rollback`. При обработке чанка все операции с данным чанком заключаются в одну транзакцию: если

на любом этапе происходит ошибка (сетевая, логическая или исчерпание попыток LLM), выполняется `rollback`, и все изменения, внесённые этим чанком, отменяются. Уже закоммиченные чанки не затрагиваются.

7.2 Работа с пассажами

Каждый пассаж — это узел `Passage` с полем `text` и уникальным `id`. При добавлении пассажа используется `MERGE`, что гарантирует идемпотентность: повторный вызов с тем же идентификатором не создаст дубликат. Связь сущности с пассажем (`MENTIONED_IN`) также создаётся через `MERGE`, а не `CREATE`, чтобы предотвратить появление множественных рёбер при многократном вызове `add_entity` для одной и той же пары (сущность, пассаж).

7.3 Отношения и `based_on`

Отношения представлены рёбрами типа `RELATION` с атрибутом `type` и списком `based_on`. При добавлении отношения с новым пассажем выполняется `MERGE` с условной логикой:

- Если ребро создаётся впервые (`ON CREATE`), `based_on` инициализируется списком из одного элемента — текущего `passage_id`.
- Если ребро уже существует (`ON MATCH`), проверяется, нет ли уже такого `passage_id` в списке; если нет — он добавляется.

Такой подход обеспечивает, что каждое отношение хранит все источники, из которых оно было извлечено, и при этом не замусоривается дубликатами.

7.4 Векторный поиск

Метод `vector_search` вычисляет эмбединг запроса с помощью `EmbeddingClient`, затем вызывает встроенную процедуру `Memgraph vector_search.search`, передавая имя индекса, желаемое количество результатов и вектор. Процедура возвращает узлы и расстояния; результаты преобразуются в список словарей с именем, типом и `score`. Индекс использует библиотеку `USearch`, которая обеспечивает быстрое приближённое нахождение ближайших соседей, критичное для скорости ответа.

7.5 Снепшоты и очистка

Снепшоты `Memgraph` — это резервные копии всего состояния графа на диск. Метод `snapshot` вызывается автоматически после обработки текста, при явном вызове `close()` экземпляра `GraphRAG` и при аварийном завершении процесса (через глобальный обработчик `atexit`). Механизм `graceful shutdown` регистрирует все активные экземпляры и гарантирует, что даже при неожиданной остановке (например, по `Ctrl+C`) данные не будут потеряны.

Метод `clear` полностью очищает граф: удаляет все узлы и связи командой `MATCH (n) DETACH DELETE n`. Если использовался векторный индекс, он также удаляется (`DROP VECTOR INDEX entity_embeddings`), и тут же создаётся заново. Это позволяет быстро подготовить чистое окружение для нового эксперимента без ручного вмешательства.

8 Клиенты LLM и эмбедингов

8.1 LLMClient

Взаимодействие с языковой моделью построено поверх библиотеки `openai` в режиме совместимости с локальными серверами. Клиент принимает объект `LLMConfig` (модель, URL, ключ, температура) и путь к файлу SQLite-кэша. Кэширование реализовано через таблицу `cache` с составным ключом, образованным MD5-хешем от конкатенации промпта и имени модели, и полями `response` и `timestamp`. При запросе сначала проверяется кэш; при попадании ответ возвращается мгновенно. При промахе выполняется HTTP-вызов, и успешный ответ сохраняется с текущей временной меткой.

Для повышения отказоустойчивости реализован механизм повторных попыток: при сетевой ошибке или таймауте делается до `max_retries` попыток с экспоненциальной задержкой (начиная с `retry_delay` секунд, умноженной на 2^{attempt}). Кэш можно периодически чистить от устаревших записей методом `cleanup_cache` (по умолчанию удаляются ответы старше 1 часа), что предотвращает неконтролируемый рост базы.

При создании клиента автоматически проверяется структура таблицы, и если отсутствует колонка `timestamp` (например, после миграции со старой версии), она добавляется. Это обеспечивает обратную совместимость.

8.2 EmbeddingClient

Для вычисления текстовых эмбедингов используется библиотека `sentence-transformers` с моделью по умолчанию `all-MiniLM-L6-v2` (размерность 384). Модель загружается лениво при первом обращении к свойству `model`, что экономит память до момента реальной необходимости. Кэширование эмбедингов реализовано в отдельной таблице `embedding_cache` с ключом на основе MD5-хеша от текста и опциональных параметров `prompt_name` и `prompt`.

Два режима работы:

- Одиночный (`embed`) — для одного текста: при промахе кодирует текст с нормализацией и точностью `float32`, сохраняет в кэш и возвращает.
- Пакетный (`embed_batch`) — для списка текстов: сначала проверяется кэш для каждого; отсутствующие собираются и кодируются одним вызовом `model.encode` с параметром `batch_size`. Затем результаты сохраняются в кэш. Вся работа с базой в этом методе ведётся в одном соединении, что снижает накладные расходы.

Пакетная обработка существенно ускоряет вычисление эмбедингов для множества сущностей, так как используется преимущество векторизованных вычислений модели и сокращается количество раундов обращения к базе.

9 Утилиты безопасного парсинга JSON

Одной из проблем при использовании LLM является нестабильность формата ответа. Модель может добавить пояснительный текст до или после JSON, обернуть

его в markdown-разметку (“`json` ... “) или вовсе выдать некорректный JSON. Функция `safe_parse_json` реализует устойчивый подход:

1. Ищет в ответе блок, начинающийся с “`json` и заканчивающийся “. Извлекает содержимое.
2. Если блок не найден, ищет первую открывающую скобку (`[` или `{`) и соответствующую последнюю закрывающую. Это позволяет извлечь JSON, даже если вокруг него есть другой текст.
3. Пытается разобрать полученную строку с помощью `json.loads`. При неудаче возвращает `None`.

Такой эвристический подход покрывает подавляющее большинство реальных ответов современных LLM (Qwen, Llama, Mistral) и делает пайплайн значительно более надёжным без необходимости тонкой настройки модели.

10 Особенности реализации и принятые решения

В этом разделе сведены ключевые технические и методологические решения, уже рассмотренные в предыдущих разделах, с акцентом на их взаимосвязь.

- **Потокобезопасность.** Блокировка в `MemgraphStore` сериализует доступ к графу, а создание отдельных `LLMClient` для каждого потока в GoT устраняет гонки в HTTP-клиенте. Это компромисс между производительностью и простотой реализации, оправданный для прототипа.
- **Graceful shutdown.** Глобальная регистрация экземпляров и вызов `snapshot()` через `atexit` гарантирует сохранность данных даже при аварийном завершении. Явное закрытие экземпляра освобождает ресурсы и удаляет его из списка.
- **Устойчивость к ошибкам модели.** Сочетание fallback-промптов, `safe_parse_json` и повторных попыток с экспоненциальной задержкой позволяет системе работать даже с нестабильными или слабыми LLM.
- **Итеративное разрешение конфликтов.** Отказ от предварительного сбора всех конфликтов в пользу итеративного перестроения рабочего списка предотвращает ошибки, связанные с удалением уже удалённых объектов.
- **Кэширование на всех уровнях.** LLM-запросы кэшируются по полному промпту, эмбединги — по тексту. Это ускоряет повторные запуски и отладку, а также снижает нагрузку на модель.
- **Атомарность операций с чанком.** Каждый чанк обрабатывается в рамках одной транзакции: либо все извлечённые данные попадают в граф, либо ни одно. Это предотвращает «грязное» состояние при частичных сбоях.

Принятые решения делают систему надёжной, гибкой и удобной для экспериментов, не перегружая её излишней сложностью.

11 Заключение

Представленная система демонстрирует полный цикл построения графа знаний из неструктурированного текста и последующего использования этого графа для ответа на вопросы. Архитектура модульная, что упрощает замену отдельных компонентов (например, переход на другую LLM или графовую СУБД). Особое внимание уделено практическим аспектам: кэшированию дорогих операций, обработке ошибок модели, потокобезопасности и сохранности данных.

Потенциальные направления развития включают:

- Более интеллектуальное разрешение конфликтов с учётом временных меток и источников.
- Поддержка нескольких пользователей и сессий.
- Интеграция с онтологиями для улучшения типизации сущностей.
- Оптимизация параллельной работы с графом (пул соединений).
- Веб-интерфейс для демонстрации.

Проект может служить основой для исследовательских прототипов в области NLP и графовых баз данных.